# Sombreamento especular em *Gaussian Opacity Fields*: integração sem alterar o rasterizador e avaliação controlada em objetos brilhantes

[Autor(a) 1][1*], [Autor(a) 2][1]

[1][Departamento, Universidade Federal Fluminense, Niterói, Brasil (confirmar)].

*Autor(a) correspondente. E-mail: [preencher]

Versão v0 — *preprint* para revisão interna; não citar.

**Métodos de reconstrução baseados em Gaussianas 3D representam a aparência dependente da vista por harmônicos esféricos de baixa ordem, o que borra reflexos especulares e mistura iluminação com material. O *Gaussian Opacity Fields* (GOF) acrescenta geometria contínua e extração de malha, mas herda esse modelo de cor; o GaussianShader introduz uma função de sombreamento com tom especular, rugosidade e mapa de ambiente, mas sem o campo de opacidade do GOF. Investigamos se esse sombreamento pode ser acoplado ao GOF sem modificar o rasterizador e quanto isso altera qualidade e custo. A cor de cada Gaussiana é calculada fora do rasterizador (difuso mais Fresnel–Schlick vezes luz pré-filtrada) e entregue pelo caminho de cor pré-computada, que já é diferenciável; com tom especular nulo e Fresnel desligado, a imagem coincide com a do GOF (diferença máxima abaixo de $10^{-5}$), o que torna a ablação controlada. Na cena real Truck, o termo especular eleva o PSNR em 0,41 dB (IC 95%: 0,29 a 0,53; 28 de 32 vistas). Em três objetos brilhantes do NeRO, a configuração herdada do Truck ganha 2,0 a 2,5 dB sobre o GOF em dois objetos e nada no mais espelhado; ajustando três hiperparâmetros, o método empata com o GaussianShader nesse**

**objeto (−0,01 dB) e o supera nos outros dois (+1,37 e +0,58 dB), gastando de 2,7 a 4,0 vezes menos energia de GPU e metade da memória. Os números vêm de uma semente por configuração, e o ajuste foi escolhido em um dos objetos de teste; repetir sementes e separar os ajustes são os próximos passos.**

# 1 Introdução

A síntese de novas vistas e a reconstrução 3D a partir de fotografias calibradas passaram, em poucos anos, de representações implícitas por redes neurais (*1*) para representações explícitas por Gaussianas 3D rasterizadas em tempo real (*2, 3*). Nessas representações, a cor de cada Gaussiana é uma função da direção de observação escrita em harmônicos esféricos (SH) de grau baixo. Essa escolha é eficiente e suficiente para superfícies foscas, mas tem duas consequências conhecidas em superfícies brilhantes. Primeiro, um reflexo especular nítido é um sinal de alta frequência angular, e uma expansão em poucos SH só consegue representá-lo borrado (*4, 5*). Segundo, a radiância observada mistura propriedades do material e a iluminação da cena: a luz fica "assada" na cor de cada primitiva, o que impede separar material de iluminação e degrada o ajuste em objetos refletores (*6, 7*). Reflexos também violam a consistência multivista em que se apoia a reconstrução geométrica, o que torna objetos espelhados um caso difícil tanto para a aparência quanto para a geometria (*8, 9*).

Duas linhas de trabalho atacam partes desse problema. A primeira busca geometria fiel a partir de Gaussianas: alinhamento das primitivas a superfícies (*10*), discos 2D com interseção raio–disco (*11*) e, no caso que nos interessa, o *Gaussian Opacity Fields* (GOF) (*12*), que obtém a superfície diretamente como conjunto de nível da opacidade das Gaussianas, com normais dadas pelo plano de interseção raio–Gaussiana e extração de malha por *Marching Tetrahedra*. O GOF, porém, mantém o modelo de cor em SH do 3DGS. A segunda linha acrescenta sombreamento fisicamente inspirado às Gaussianas. O GaussianShader (*6*) define, para cada Gaussiana, uma cor difusa, um tom especular, uma rugosidade e uma cor residual, e calcula a luz especular a partir de um mapa de ambiente aprendido e pré-filtrado; métodos posteriores acrescentam sombreamento diferido, oclusão e iluminação indireta (*13–16*). Esses métodos tratam a geometria com aproximações próprias de normal e, em geral, não partem do campo de opacidade do GOF.

Este trabalho investiga a junção dessas duas linhas em sua forma mais simples: acoplar o sombreamento do GaussianShader ao GOF sem modificar o rasterizador do GOF. A pergunta de pesquisa é se essa integração (i) reproduz exatamente o GOF quando o termo especular é nulo, condição necessária para que uma comparação "com e sem especular" isole o efeito do termo; (ii) melhora a síntese de vistas em cena real e em objetos brilhantes; e (iii) faz isso a um custo computacional menor que o do GaussianShader, numa GPU de consumo com 6 GB.

Um levantamento bibliográfico feito no início deste estudo indicou que os ingredientes isolados não são novos: a parametrização por tom especular, rugosidade e cor residual vem do GaussianShader (*6*), a reparametrização pela direção de reflexão vem do Ref-NeRF (*4*) e a pré-filtragem da iluminação segue a aproximação *split-sum* (*17, 18*). Por isso declaramos as contribuições de forma restrita:

1. uma integração do sombreamento especular ao GOF que não altera o rasterizador CUDA nem o caminho de extração de malha, cuja equivalência exata com o GOF é verificada por teste automatizado (Seção 3);

2. uma avaliação controlada, com estatística pareada por vista, em uma cena real (Truck) e em três objetos brilhantes (NeRO), comparando GOF, GOF com sombreamento em duas configurações e GaussianShader, com tempo, energia de GPU e memória medidos em cada treino (Seções 4 e 5);

3. uma análise do mecanismo pelo qual a configuração afeta o termo especular, a partir das distribuições de material aprendidas, que refuta uma das hipóteses do nosso próprio diagnóstico inicial (Seção 5.4);

4. o registro de modos de falha silenciosos dessa integração (estado do material perdido na poda, na serialização e no *checkpoint*; gradiente infinito nos polos do mapa de ambiente) e dos testes que os detectam (Material Suplementar, Seção S3).

# 2 Trabalhos relacionados

**Geometria a partir de Gaussianas.** O 3D Gaussian Splatting (3DGS) (*2*) otimiza Gaussianas anisotrópicas por rasterização diferenciável e atinge renderização em tempo real, mas suas pri-

mitivas desconexas não definem uma superfície. O SuGaR (*10*) regulariza as Gaussianas para se alinharem à superfície e extrai a malha por reconstrução de Poisson. O 2D Gaussian Splatting (*11*) troca os elipsoides por discos orientados e usa interseção raio–disco, perda de distorção de profundidade e consistência de normais. O GOF (*12*) deriva um campo de opacidade da renderização volumétrica por interseção raio–Gaussiana, aproxima a normal pelo plano de interseção e extrai a malha por *Marching Tetrahedra* sobre uma grade induzida pelas próprias Gaussianas, sem fusão TSDF nem Poisson; sua implementação inclui o filtro 3D do Mip-Splatting (*19*). Trabalhos seguintes rasterizam profundidade e normal de Gaussianas gerais (*20*), acoplam um campo de distância com sinal (*21*), extraem a malha de forma diferenciável durante o treino (*22*), ou reformulam a renderização por campos de geometria (*23*) e por traçado de raios de partículas (*24*). Em superfícies especulares, a variação fotométrica entre vistas pode fazer a geometria colapsar para dentro do objeto (*9*).

**Aparência especular e renderização inversa.** O Ref-NeRF (*4*) reparametriza a radiância pela direção de reflexão e introduz uma codificação direcional integrada que atenua bandas altas conforme a rugosidade. O NeRO (*8*) reconstrói geometria e BRDF de objetos refletores com a aproximação *split-sum* e propõe o conjunto Glossy Synthetic usado aqui; o nvdiffrec (*18*) formula o *split-sum* de forma diferenciável. Sobre Gaussianas, o GaussianShader (*6*) introduz a função de sombreamento que adotamos, com normal pelo eixo mais curto da Gaussiana; o GIR (*14*) também usa o menor autovetor como normal e acrescenta iluminação indireta por voxels; o GS-IR (*13*) regulariza a normal pela derivada da profundidade e modela oclusão por cozimento (*baking*). Outros métodos adotam sombreamento diferido (*7, 15*), combinam sombreamento direto e diferido (*25, 26*), tratam objetos brilhantes especificamente (*27, 28*), guiam a geometria por campos de distância (*29, 30*) ou pela própria geometria (*31*), e variam o material dentro de cada Gaussiana (*32*). Uma terceira onda trata a iluminação indireta de forma explícita, por traçado de raios em Gaussianas 2D (*16, 33*), por traçado de caminhos (*34*), por radiosidade (*35*), por consistência radiométrica (*36*) ou por iluminação global diferida (*37*). Há ainda abordagens que geram materiais por difusão (*38*) e que operam com poucas vistas (*39*).

**Tabela 1**: **Comparação qualitativa com trabalhos próximos.** Descrições tiradas dos resumos e textos dos trabalhos citados; "—" indica que o aspecto não é tratado pelo método.

| Método | Primitiva e renderização | Modelo de aparência | Normal do sombreamento | Luz indireta |
|---|---|---|---|---|
| 3DGS (*2*) | Gaussiana 3D, *splatting* | SH por Gaussiana | — | assada na cor |
| 2DGS (*11*) | disco 2D, interseção raio–disco | SH por Gaussiana | — | assada na cor |
| GOF (*12*) | Gaussiana 3D, interseção raio–Gaussiana | SH por Gaussiana | — | assada na cor |
| GaussianShader (*6*) | Gaussiana 3D, *splatting* | difuso + tom especular × luz pré-filtrada + residual | eixo mais curto + resíduo | residual aprendido |
| GS-IR (*13*) | Gaussiana 3D, *splatting* | BRDF física | derivada da profundidade | oclusão cozida |
| Ref-Gaussian (*15*) | Gaussiana 2D, sombreamento diferido | BRDF física por pixel (*split-sum*) | propagação ciente do material | inter-reflexão por Gaussianas |
| IRGS (*16*) | Gaussiana 2D, traçado de raios | equação de renderização completa | da primitiva 2D | traçado de raios |
| Este trabalho | Gaussiana 3D do GOF (inalterada) | difuso + Fresnel–Schlick × luz pré-filtrada (+ residual SH) | eixo mais curto | — |

**Posicionamento.** A Tabela 1 resume as diferenças relevantes. Nosso método é deliberadamente mais simples que os da terceira onda: não modela iluminação indireta nem visibilidade, e sombreia cada Gaussiana uma vez por vista, como o GaussianShader. O que o distingue é manter intactos a primitiva, o rasterizador e a extração de malha do GOF, o que permite atribuir qualquer diferença em relação ao GOF ao termo de sombreamento.

# 3 Método

## 3.1 Preliminares: renderização no GOF

Cada Gaussiana $i$ tem centro $\mathbf{x}_i$, covariância $\Sigma_i = R_i S_i S_i^\top R_i^\top$ (rotação $R_i$ e escalas $S_i$), opacidade $\alpha_i$ e coeficientes SH de cor. O GOF avalia, para cada raio de pixel, o valor máximo $\mathcal{G}_i$ da Gaussiana ao longo do raio (interseção raio–Gaussiana) e compõe as cores na ordem de profundidade (*12*):

$$C = \sum_i \mathbf{c}_i \, \alpha_i \mathcal{G}_i \prod_{j<i} \left(1 - \alpha_j \mathcal{G}_j\right). \tag{1}$$

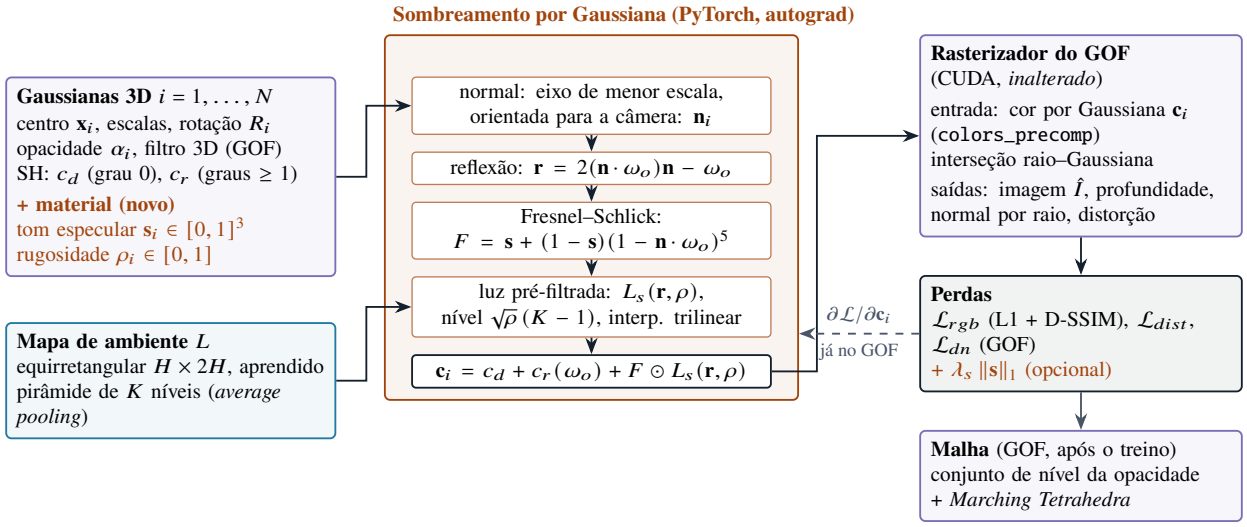



**Figura 1**: **Visão geral do método.** O sombreamento (laranja) calcula, em PyTorch, uma cor $\mathbf{c}_i$ por Gaussiana e por vista a partir dos atributos de material e de um mapa de ambiente aprendido. Essa cor entra no rasterizador do GOF pelo caminho de cor pré-computada, que já existia e já devolvia $\partial\mathcal{L}/\partial\mathbf{c}_i$; o restante do gradiente é obtido por diferenciação automática. Rasterizador, perdas geométricas e extração de malha são os do GOF, sem modificação.

No GOF, $\mathbf{c}_i$ é a avaliação dos SH na direção da câmera para o centro da Gaussiana. As mesmas quantidades produzem mapas de profundidade e de normal, usados em uma perda de distorção de profundidade e em uma perda de consistência entre a normal renderizada e a normal derivada da profundidade. A superfície final é um conjunto de nível do campo de opacidade, extraído por *Marching Tetrahedra*. Nada disso é alterado neste trabalho: mudamos apenas o valor de $\mathbf{c}_i$ na Eq. 1.

## 3.2 Função de sombreamento

Seguindo o GaussianShader (*6*), cada Gaussiana recebe um tom especular $\mathbf{s}_i \in [0,1]^3$ e uma rugosidade $\rho_i \in [0,1]$ (ambos parametrizados por sigmoide e inicializados em 0,05 e 0,70, isto é, um material quase difuso). Sendo $\omega_o$ a direção unitária do centro da Gaussiana para a câmera, a cor é

$$\mathbf{c}_i(\omega_o) = \max\Big(\underbrace{c_{d,i} + c_{r,i}(\omega_o)}_{\text{SH do GOF}} + F(\mathbf{n}_i\cdot\omega_o, \mathbf{s}_i) \odot L_s(\mathbf{r}_i, \rho_i),\ 0\Big). \tag{2}$$

**Termos difuso e residual.** Em vez de criar um tensor novo para a cor residual, reaproveitamos a expansão SH que cada Gaussiana do GOF já possui: o grau 0 é constante em $\omega_o$ e faz o papel do difuso $c_d$, e os graus $\geq 1$ formam a parte dependente da vista, $c_r$. O GaussianShader trata $c_d$ e $c_r$ como termos separados, com $c_r$ em SH de terceira ordem (*6*); nossa leitura é uma reinterpretação que mantém a memória e garante a equivalência da Seção 3.5. **Em todos os experimentos deste artigo usamos SH de grau 0**, por restrição de memória da GPU de 6 GB; portanto $c_r \equiv 0$ e a dependência da vista vem apenas do termo especular.

**Normal.** Como no GaussianShader e no GIR (*6, 14*), a normal de sombreamento é o eixo de menor escala da Gaussiana, $\mathbf{n}_i = R_i\,\mathbf{e}_k$ com $k = \arg\min_j S_{i,jj}$, com o sinal escolhido de modo que $\mathbf{n}_i \cdot \omega_o > 0$. O resíduo aprendível de normal do GaussianShader está implementado mas desligado nos experimentos. Note que essa normal é por Gaussiana, enquanto a normal do GOF é por raio; as duas coincidem quando a Gaussiana é achatada, regime que as perdas geométricas do GOF favorecem, mas não são a mesma quantidade (Seção 7).

**Reflexão e Fresnel.** A direção de consulta à iluminação é a reflexão $\mathbf{r}_i = 2(\mathbf{n}_i \cdot \omega_o)\mathbf{n}_i - \omega_o$ (*4*). O fator especular usa a aproximação de Schlick (*40*) com o tom especular no papel da refletância em incidência normal:

$$F(\cos\theta, \mathbf{s}) = \mathbf{s} + (\mathbf{1} - \mathbf{s})(1 - \cos\theta)^5, \qquad \cos\theta = \mathbf{n} \cdot \omega_o. \tag{3}$$

Este termo não existe na equação do GaussianShader, em que o tom multiplica a luz diretamente (*6*); ele é uma escolha nossa, que faz bordas em ângulo rasante refletirem mais. Como $F \to 1$ em ângulo rasante qualquer que seja $\mathbf{s}$, a equivalência exata com o GOF exige desligar o Fresnel (Seção 3.5).

**Luz especular pré-filtrada.** A iluminação é um mapa de ambiente equirretangular aprendível $L$ de $H \times 2H$ texels, inicializado em 0,5 com ruído gaussiano de desvio 0,02 (o ruído evita partir de uma iluminação uniforme, sob a qual a rugosidade não tem gradiente). A cada passo construímos uma pirâmide de $K = 6$ níveis por média $2 \times 2$ (*average pooling*) e lemos $L_s(\mathbf{r}, \rho)$ no nível contínuo $\ell = \sqrt{\rho}\,(K - 1)$, com interpolação bilinear dentro do nível e linear entre níveis. Isso é uma aproximação barata da pré-filtragem do *split-sum* (*17, 18*): o filtro é uma caixa, não a convolução pelo lóbulo GGX (*41*) usada nesses trabalhos. Uma consequência importante, discutida

na Seção 5.4, é que $\rho$ indexa um nível de desfoque cuja largura angular depende de $H$, e não é uma rugosidade física calibrada. As coordenadas $(u, v)$ do mapa vêm de atan2 e arccos da direção $\mathbf{r}$; afastamos a direção dos polos por $\varepsilon = 10^{-4}$, porque o gradiente dessas funções diverge exatamente nos polos e, num treino real, bastou uma Gaussiana perto de um polo para levar a perda a `NaN`.

**Referencial da iluminação.** Em todos os experimentos o mapa é fixo no referencial do mundo, que é o caso físico de uma câmera orbitando um objeto parado sob luz fixa. A implementação também permite indexar o mapa pela reflexão em coordenadas da câmera, pensada para captura em mesa giratória; a justificativa está no Material Suplementar (Seção S4) e esse modo *não* foi avaliado empiricamente.

## 3.3 Integração sem alterar o rasterizador

O rasterizador do GOF, herdado do 3DGS, aceita uma cor RGB por Gaussiana já calculada (`colors_precomp`) no lugar dos SH e devolve o gradiente da perda em relação a essa cor. Calculamos a Eq. 2 em PyTorch e a entregamos por esse caminho (Figura 1). Todo o gradiente a jusante (tom, rugosidade, rotação e escalas pela normal, e o próprio mapa de ambiente) é obtido por diferenciação automática, sem derivadas escritas à mão e sem código CUDA novo. O ramo especular é ligado a partir da iteração 3.000; antes disso o modelo renderiza como o GOF.

## 3.4 Perdas

A perda é a do GOF, $\mathcal{L}_{rgb} + \lambda_{dist}\mathcal{L}_{dist} + \lambda_{dn}\mathcal{L}_{dn}$, com $\mathcal{L}_{rgb} = 0{,}8\,\mathcal{L}_1 + 0{,}2\,\mathcal{L}_{\text{D-SSIM}}$ (*2*), $\lambda_{dist} = 100$ e $\lambda_{dn} = 0{,}05$, acrescida opcionalmente de uma esparsidade do tom especular, $\lambda_s\,\mathbb{E}_i\|\mathbf{s}_i\|_1$. Essa esparsidade codifica a hipótese de que poucas Gaussianas são especulares, útil para reduzir a ambiguidade entre albedo claro e reflexo em cenas foscas. Ela não é a perda homônima do GaussianShader, que atua sobre a opacidade (*6*).

## 3.5 Equivalência verificável

Para que a diferença entre "GOF" e "GOF com sombreamento" meça apenas o termo especular, verificamos por teste automatizado, na GPU, duas igualdades: (a) renderizar com as cores calculadas

em PyTorch pelo caminho pré-computado coincide com renderizar pelos SH avaliados no CUDA do GOF; e (b) com $\mathbf{s} \approx 0$ e Fresnel desligado, a Eq. 2 reproduz o GOF. Nos dois casos a diferença máxima por pixel é menor que $10^{-5}$. Um terceiro teste confirma que, com o Fresnel ligado, a igualdade (b) deixa de valer, como esperado pela Eq. 3. Esses testes fazem parte de uma suíte de 36 testes que passam na versão usada nos experimentos (Material Suplementar, Seção S3).

# 4 Protocolo experimental

## 4.1 Desenho

O desenho separa três comparações de natureza diferente. **(C1) Efeito do termo especular:** GOF contra GOF+BRDF com o mesmo código e os mesmos hiperparâmetros, diferindo só pela ativação do sombreamento; é a única comparação causal no sentido estrito. **(C2) Efeito da configuração:** GOF+BRDF com a configuração usada no Truck contra GOF+BRDF ajustado; mede o efeito conjunto de três hiperparâmetros alterados ao mesmo tempo. **(C3) Comparação de sistemas:** GOF+BRDF ajustado contra GaussianShader; os dois diferem em código, primitiva, normal, iluminação e hiperparâmetros, e a comparação informa o desempenho relativo, não a causa.

## 4.2 Dados

**Truck.** Cena externa do Tanks and Temples (*42*), com 251 fotografias e poses estimadas por *structure-from-motion* (*43*). Treinamos em resolução reduzida por 2 (978×546) e usamos como teste uma a cada oito imagens (32 vistas), como no protocolo do 3DGS. A inicialização usa os pontos esparsos do SfM.

**NeRO Glossy Synthetic.** Objetos sintéticos refletores renderizados sob iluminação de ambiente real (*8*). Escolhemos três dos oito objetos para caber no orçamento de GPU (cada treino leva de 2 a 6 h), cobrindo do quase espelho ao material mais difuso: *bell* (metal dourado quase espelhado), *teapot* (metal prateado com curvatura variada) e *cat* (corpo perolado sobre base metálica). Usamos a divisão oficial em 96 vistas de treino e 32 de teste, a 800×800, com o canal alfa composto sobre fundo branco. Os dados foram convertidos pelo script do GaussianShader e, para o GOF,

**Tabela 2**: **Configurações avaliadas.** Hiperparâmetros não listados seguem os valores-padrão do GOF. O GaussianShader usa `--brdf_dim 0 --sh_degree -1 --lambda_predicted_normal 2e-1 --brdf_env 512`, como no repositório dos autores, com iluminação em *cubemap* e regras próprias para normal e ativação dos termos.

| | GOF | GOF+BRDF (config. Truck) | GOF+BRDF ajustado | GaussianShader |
|---|---|---|---|---|
| Sombreamento especular | não | sim | sim | sim |
| Mapa de ambiente | — | 128×256 | 512×1024 | *cubemap* $6 \times 512^2$ |
| Esparsidade do tom $\lambda_s$ | — | 0,001 | 0 | — |
| Consistência de normal a partir de | 15.000 | 15.000 | 3.000 | (própria) |
| Especular a partir de | — | 3.000 | 3.000 | (própria) |
| Fresnel–Schlick | — | sim | sim | não |
| Grau SH da cor | 0 | 0 | 0 | −1 (cor pelo ramo BRDF) |
| Iterações | 30.000 | 30.000 | 30.000 | 30.000 |

acrescentamos o campo de abertura horizontal da câmera exigido pelo leitor de cenas. Os dois métodos partem de 100 mil pontos aleatórios. Verificamos que as imagens de referência gravadas pelos dois métodos são idênticas pixel a pixel.

## 4.3 Configurações

A Tabela 2 lista as configurações. Todas as variantes do GOF usam SH de grau 0, 30 mil iterações e o mesmo código; o GaussianShader usa o comando indicado no repositório dos autores para cenas sintéticas.

**Como o ajuste foi escolhido.** Com a configuração do Truck, o GOF+BRDF ficou 5 dB abaixo do GaussianShader na *bell*. O diagnóstico apontou quatro causas possíveis: rugosidade aprendida alta num objeto espelhado, mapa de ambiente de baixa resolução, esparsidade empurrando o tom especular para zero e regularização de normal iniciada tarde (15 mil iterações) em relação ao ramo especular (3 mil). As três últimas foram alteradas juntas e conferidas em treinos curtos de 7.002 iterações na *bell*, com PSNR do avaliador interno de 23,75 (GOF), 24,36 (config. Truck) e 27,44 dB (ajustado), antes dos treinos completos nas três cenas. Como a *bell* é também cena de teste, o ganho nela está sujeito a viés de seleção; *cat* e *teapot* não participaram da escolha e funcionam como evidência fora da amostra para o ajuste. A primeira causa foi testada depois, pelas distribuições de material (Seção 5.4), e não se confirmou.

## 4.4 Métricas

Medimos PSNR, SSIM (*44*) e LPIPS com rede VGG (*45*) sobre as imagens de teste gravadas por cada método, com o mesmo código para todos (a implementação de métricas do GOF), na imagem inteira. No NeRO o fundo branco entra no cálculo e eleva os valores absolutos de todos os métodos por igual; as comparações são relativas.

## 4.5 Análise estatística

A unidade de análise é a vista de teste. Para cada par de métodos calculamos a diferença por vista e reportamos a média, um intervalo de confiança de 95% por *bootstrap* pareado das diferenças (10 mil reamostragens) (*46*), o valor-$p$ bilateral do teste de postos sinalizados de Wilcoxon (*47*), o tamanho de efeito $d_z$ de Cohen e o número de vistas em que cada método é melhor. Com 32 vistas e todas as diferenças do mesmo sinal, o menor $p$ possível do teste exato é $2/2^{32} \approx 4{,}7 \times 10^{-10}$. Ao todo reportamos 24 pares em 3 métricas (72 testes); o limiar de Bonferroni para $\alpha = 0{,}05$ seria $6{,}9 \times 10^{-4}$, e todas as diferenças que tratamos como significativas têm $p \leq 1{,}0 \times 10^{-4}$; os dois efeitos que tratamos como nulos são o PSNR da *bell* em C1 ($p = 0{,}59$) e em C3 ($p = 0{,}71$).

Essa análise trata cada modelo treinado como fixo: o intervalo descreve a incerteza devida à amostra de vistas, não a variação entre treinos. O treino do GOF não é reprodutível bit a bit, porque o rasterizador acumula gradientes com operações atômicas cuja ordem varia; por isso, com uma semente por configuração, consideramos não interpretáveis diferenças médias de PSNR abaixo de cerca de 0,1 dB, mesmo quando o intervalo exclui o zero.

## 4.6 Custo

Todos os treinos rodaram um por vez na mesma máquina, com uma GPU NVIDIA RTX 3050 de 6 GB (CUDA 11.8, PyTorch 2.0.1); o GaussianShader rodou em ambiente separado, conforme seu repositório. No NeRO, um processo auxiliar amostrou o `nvidia-smi` a cada 2 s durante o treino. Reportamos o tempo de parede do treino, a potência média da placa, a energia como potência média vezes tempo (apenas a GPU, sem CPU nem fonte; ver (*48*) sobre a importância de reportar esse custo) e a memória de vídeo máxima ocupada segundo o `nvidia-smi`, que inclui o cache do alocador e o contexto CUDA e é, portanto, um limite superior da memória necessária. No Truck,

medido antes desse instrumento, reportamos o tempo de parede dos registros de treino e o pico de memória alocada pelo PyTorch, que é outra medida e não deve ser comparada diretamente com a do NeRO.

# 5 Resultados

## 5.1 Cena real: Truck

A Tabela 3 mostra a comparação controlada C1 no Truck. O termo especular aumenta o PSNR em 0,41 dB (IC 95%: +0,29 a +0,53), com ganho em 28 das 32 vistas, e aumenta o SSIM em todas as vistas. O LPIPS, ao contrário, piora levemente (+0,0025) em 30 das 32 vistas. O ganho é distribuído pelas vistas (mediana +0,38 dB); o maior é de 1,66 dB, na vista mostrada na Figura 2, e nas quatro vistas em que o GOF fica à frente a diferença não passa de 0,14 dB. Na vista de maior ganho, a redução do erro não se concentra numa região especular: no terço superior, no meio e na parte de baixo da imagem, o GOF+BRDF erra menos em 56 a 58% dos pixels, e há também regiões em que o erro aumenta, como uma mancha no chão à direita (Figura 2). O número de Gaussianas finais é praticamente o mesmo (2,09 e 2,08 milhões), o tempo de treino passa de 6 h 51 min para 8 h 22 min (+22%), e o pico de memória alocada do GOF+BRDF foi de 3,99 GB (no GOF, cerca de 2,9 GB, medido até a iteração 6.777). Para comparação, o GaussianShader relata ganho médio de +0,19 dB sobre o 3DGS no Tanks and Temples, cujos objetos são predominantemente difusos, e de +0,04 dB no próprio Truck (Tabela 6 de (*6*)); o efeito que medimos é maior, mas a comparação entre trabalhos não é direta, porque a base (3DGS ou GOF), a resolução e o protocolo diferem. Comparações no Truck com o GaussianShader e com outra configuração do GOF, treinados em outra máquina e com outros hiperparâmetros, estão na Tabela S1 e não são controladas.

## 5.2 Objetos brilhantes: NeRO

A Tabela 4 e a Figura 4 trazem as métricas médias; a Tabela 5 e a Figura 5, as diferenças pareadas.

**C1, termo especular com a configuração do Truck.** No *cat* e no *teapot* o termo especular rende +2,49 dB (IC: +2,27 a +2,69) e +2,00 dB (+1,61 a +2,40) sobre o GOF, com SSIM e LPIPS também

**Tabela 3**: **Truck: efeito do termo especular (C1).** Média das 32 vistas de teste, uma semente por configuração. Δ = GOF+BRDF − GOF; para o LPIPS, menor é melhor.

| | PSNR (dB) ↑ | SSIM ↑ | LPIPS ↓ |
|---|---|---|---|
| GOF | 25,220 | 0,8863 | 0,1338 |
| GOF+BRDF | 25,625 | 0,8894 | 0,1363 |
| Δ médio | +0,405 | +0,0031 | +0,0025 |
| IC 95% (*bootstrap*) | [+0,29; +0,53] | [+0,0024; +0,0039] | [+0,0016; +0,0032] |
| $p$ (Wilcoxon) | $6{,}4 \times 10^{-8}$ | $4{,}7 \times 10^{-10}$ | $2{,}7 \times 10^{-6}$ |
| $d_z$ (Cohen) | +1,14 | +1,43 | +1,06 |
| vistas em que GOF+BRDF é melhor | 28/32 | 32/32 | 2/32 |

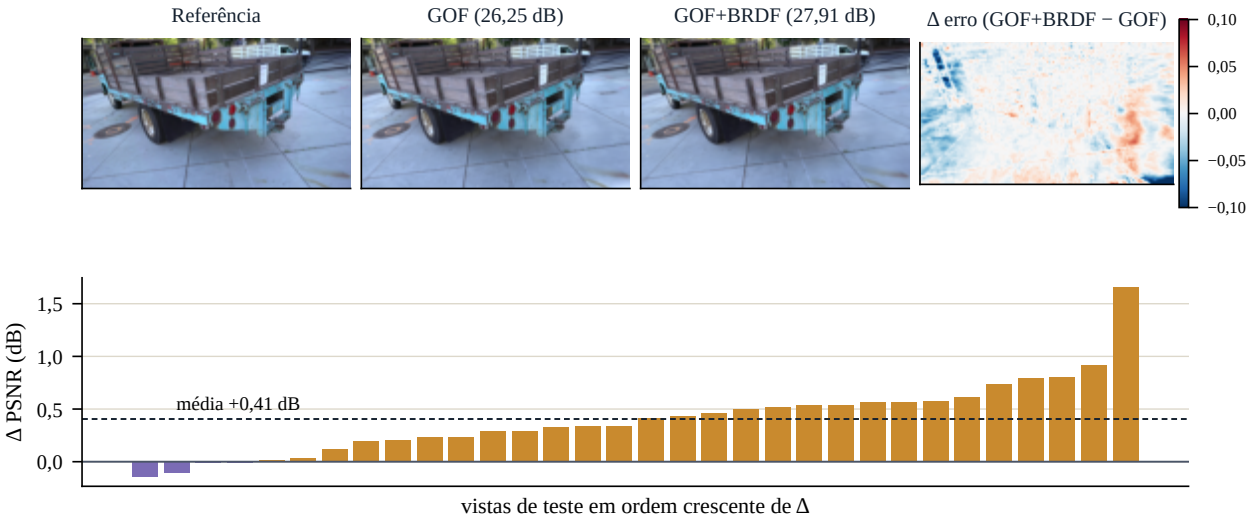



**Figura 2**: **Truck.** Acima: vista de teste com o maior ganho do termo especular (vista 24), e a diferença do erro absoluto por pixel, GOF+BRDF menos GOF (azul: o GOF+BRDF erra menos). Abaixo: diferença de PSNR por vista, ordenada; a linha tracejada é a média.

melhores. Na *bell*, o objeto mais espelhado, o efeito em PSNR é nulo (−0,04 dB; IC −0,26 a +0,18; $p = 0{,}59$), e SSIM e LPIPS pioram levemente.

**C2, efeito do ajuste.** O ajuste soma +5,02 dB na *bell*, +0,32 dB no *cat* e +1,33 dB no *teapot*, com as três métricas melhores nas três cenas. O ganho no *cat* e no *teapot*, que não participaram da escolha, indica que o ajuste não é específico da *bell*, embora seja muito menor fora dela.

**C3, comparação com o GaussianShader.** Na *bell*, os dois empatam em PSNR (−0,01 dB; IC −0,24 a +0,23; $p = 0{,}71$; o GOF+BRDF ajustado é melhor em 17 das 32 vistas), mas o

**Tabela 4**: **NeRO: métricas médias nas 32 vistas de teste.** Negrito: melhor média da cena (na *bell*, a diferença de PSNR entre o GOF+BRDF ajustado e o GaussianShader não é significativa; Tabela 5). Itálico: valores publicados pelos autores do GaussianShader para o mesmo objeto (Tabela 5 de (*6*)), como referência da nossa execução.

| Cena | Método | PSNR (dB) ↑ | SSIM ↑ | LPIPS ↓ |
|---|---|---|---|---|
| bell | GOF | 24,85 | 0,9043 | 0,1109 |
| | GOF+BRDF (config. Truck) | 24,80 | 0,8928 | 0,1159 |
| | GOF+BRDF ajustado | 29,83 | 0,9350 | 0,0867 |
| | GaussianShader | **29,84** | **0,9405** | **0,0834** |
| | *GaussianShader, publicado* | *28,08* | *0,920* | *0,097* |
| cat | GOF | 29,87 | 0,9521 | 0,0720 |
| | GOF+BRDF (config. Truck) | 32,37 | 0,9665 | 0,0541 |
| | GOF+BRDF ajustado | **32,68** | **0,9698** | **0,0491** |
| | GaussianShader | 31,32 | 0,9607 | 0,0578 |
| | *GaussianShader, publicado* | *31,81* | *0,961* | *0,056* |
| teapot | GOF | 21,08 | 0,8790 | 0,1033 |
| | GOF+BRDF (config. Truck) | 23,08 | 0,8889 | 0,0982 |
| | GOF+BRDF ajustado | **24,40** | **0,9062** | **0,0921** |
| | GaussianShader | 23,82 | 0,9021 | 0,0946 |
| | *GaussianShader, publicado* | *23,57* | *0,899* | *0,090* |

GaussianShader tem SSIM (+0,0055) e LPIPS (−0,0033) significativamente melhores. No *cat* e no *teapot*, o GOF+BRDF ajustado é melhor nas três métricas, com PSNR maior em todas as 32 vistas (+1,37 e +0,58 dB). Para verificar que o GaussianShader não foi prejudicado pela nossa execução, comparamos com os valores publicados pelos autores para os mesmos objetos (linhas em itálico na Tabela 4): nossa execução fica 1,76 dB acima do publicado na *bell*, 0,25 dB acima no *teapot* e 0,49 dB abaixo no *cat*.

**Inspeção visual.** Na Figura 3, o erro se concentra nos reflexos do ambiente. Na *bell*, sem ajuste, o ambiente refletido vira manchas com ou sem o termo especular; com o ajuste, os contornos do reflexo aparecem, como no GaussianShader. Sem o termo especular, o metal do *teapot* perde contraste e o *cat* perde o brilho perolado.

## 5.3 Custo

A Tabela 6 e a Figura 6 mostram o custo de treino. Em relação ao GOF+BRDF ajustado, o GaussianShader leva de 1,9 a 3,0 vezes o tempo, gasta de 2,7 a 4,0 vezes a energia da GPU

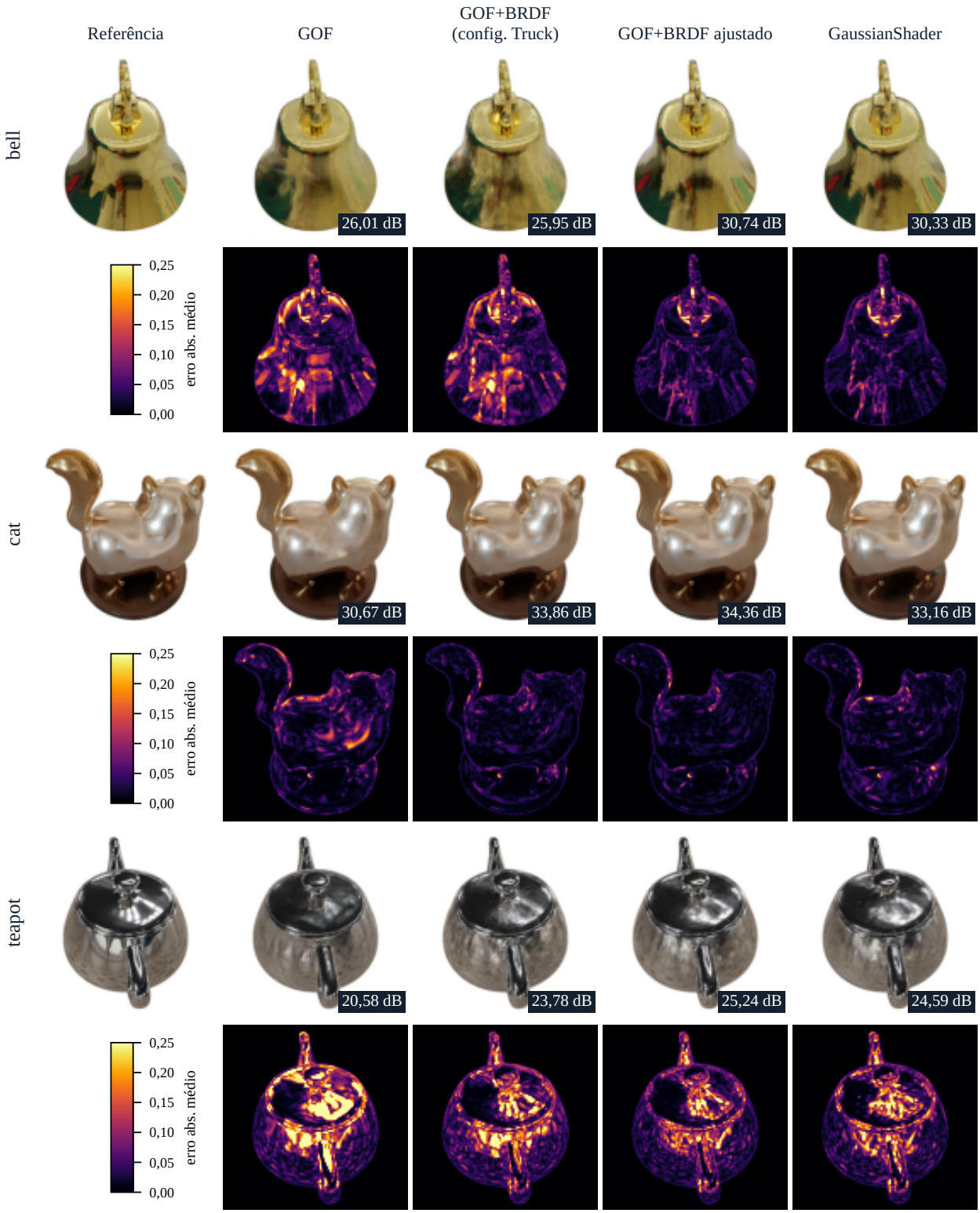



**Figura 3**: **NeRO, vista de teste 5 de cada objeto.** Linhas ímpares: referência e renders, com o PSNR da vista; linhas pares: erro absoluto médio por pixel, na mesma escala para todos os métodos.

**Tabela 5**: **NeRO: diferenças pareadas por vista.** Média e IC 95% por *bootstrap* pareado; $p$ do teste de Wilcoxon para o PSNR; última coluna: vistas em que o primeiro termo tem PSNR maior. Diferenças positivas favorecem o primeiro termo em PSNR e SSIM, e negativas em LPIPS. Estatísticas de todos os pares estão na Tabela S2.

| Cena | Contraste | ΔPSNR (dB) | ΔSSIM | ΔLPIPS | $p$ (PSNR) | vistas |
|---|---|---|---|---|---|---|
| bell | GOF+BRDF − GOF | −0,04 [−0,26; +0,18] | −0,0115 [−0,0131; −0,0100] | +0,0050 [+0,0040; +0,0060] | 0,586 | 14/32 |
| | ajustado − GOF+BRDF | +5,02 [+4,81; +5,26] | +0,0422 [+0,0388; +0,0456] | −0,0292 [−0,0312; −0,0272] | $4{,}7 \times 10^{-10}$ | 32/32 |
| | ajustado − GaussianShader | −0,01 [−0,24; +0,23] | −0,0055 [−0,0073; −0,0037] | +0,0033 [+0,0019; +0,0046] | 0,705 | 17/32 |
| cat | GOF+BRDF − GOF | +2,49 [+2,27; +2,69] | +0,0144 [+0,0137; +0,0152] | −0,0179 [−0,0189; −0,0169] | $4{,}7 \times 10^{-10}$ | 32/32 |
| | ajustado − GOF+BRDF | +0,32 [+0,21; +0,42] | +0,0034 [+0,0027; +0,0040] | −0,0050 [−0,0060; −0,0041] | $1{,}1 \times 10^{-6}$ | 29/32 |
| | ajustado − GaussianShader | +1,37 [+1,15; +1,60] | +0,0092 [+0,0077; +0,0107] | −0,0087 [−0,0100; −0,0075] | $4{,}7 \times 10^{-10}$ | 32/32 |
| teapot | GOF+BRDF − GOF | +2,00 [+1,61; +2,40] | +0,0098 [+0,0084; +0,0112] | −0,0050 [−0,0059; −0,0041] | $9{,}3 \times 10^{-10}$ | 31/32 |
| | ajustado − GOF+BRDF | +1,33 [+1,14; +1,52] | +0,0174 [+0,0160; +0,0187] | −0,0061 [−0,0068; −0,0055] | $4{,}7 \times 10^{-10}$ | 32/32 |
| | ajustado − GaussianShader | +0,58 [+0,49; +0,67] | +0,0041 [+0,0032; +0,0051] | −0,0025 [−0,0032; −0,0018] | $4{,}7 \times 10^{-10}$ | 32/32 |

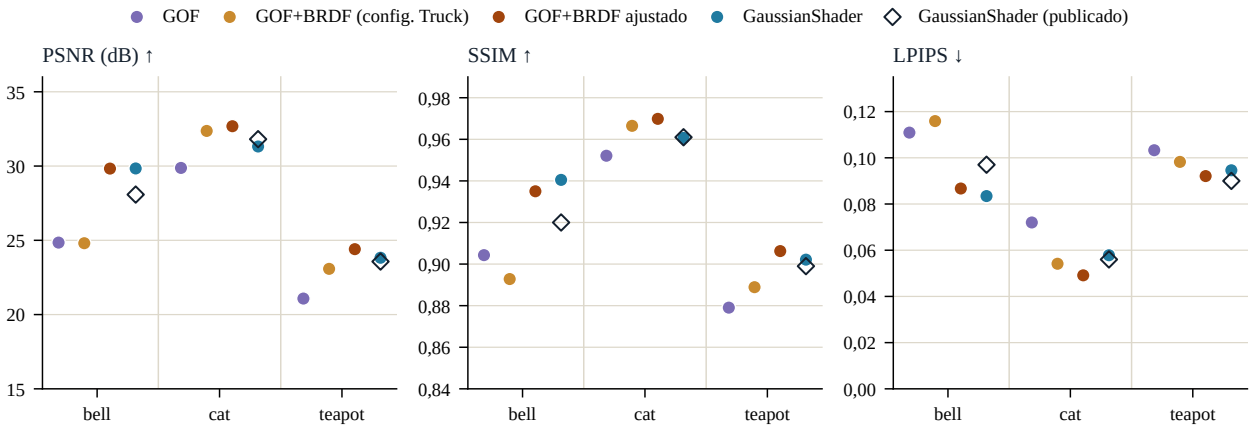



**Figura 4**: **NeRO: métricas médias por objeto.** Os losangos vazados são os valores publicados para o GaussianShader (*6*); os eixos verticais não começam em zero.

(potência média de 68 W contra cerca de 49 W) e ocupa cerca do dobro da memória, perto do limite de 6 GB da placa. O GaussianShader termina com 2,0 a 2,7 vezes mais Gaussianas. O ajuste não encarece o treino: tempo e energia do GOF+BRDF ajustado ficam iguais ou menores que os da configuração do Truck, e a memória fica entre 2,6 e 2,7 GB. A renderização das 32 vistas de teste, incluindo carga do modelo e gravação das imagens, leva cerca de 20 a 24 s nas variantes do GOF e 50 a 52 s no GaussianShader; esse tempo não é uma medida de quadros por segundo. A potência ao longo do treino está na Figura S1.

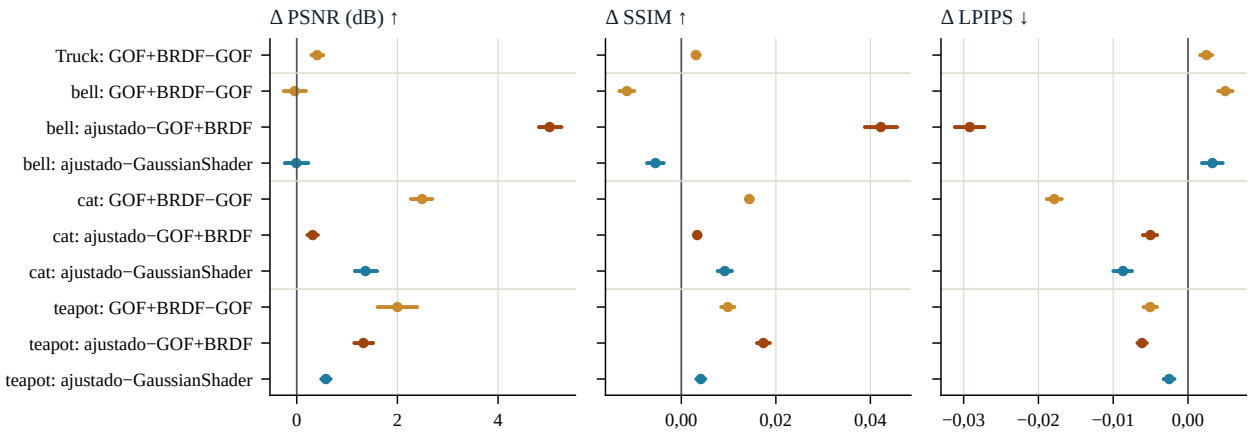



**Figura 5**: **Efeitos pareados com intervalo de confiança de 95%.** Cada linha é um contraste: termo especular com a mesma configuração (laranja-claro, incluindo o Truck), efeito do ajuste (laranja-escuro) e GOF+BRDF ajustado menos GaussianShader (azul). Para o LPIPS, valores negativos favorecem o primeiro termo.

## 5.4 Por que a configuração importa

O diagnóstico inicial da *bell* incluía a hipótese de que o modelo aprendia uma rugosidade alta demais para um objeto espelhado. As distribuições de material aprendidas (Tabela 7 e Figura 7) não sustentam essa hipótese: com o ajuste, a rugosidade mediana fica igual na *bell* (0,54 e 0,56) e *aumenta* no *cat* e no *teapot*. O que muda de forma consistente nas três cenas é o tom especular: a mediana passa de 0,12–0,23 para 0,32–0,57, e a fração de Gaussianas com tom acima de 0,1, ponderada pela opacidade, passa de 54–66% para 77–84%. Isso é o esperado ao remover a esparsidade do tom, que penaliza exatamente essa grandeza.

A resolução do mapa explica por que a mesma rugosidade pode produzir reflexos muito mais nítidos. O nível lido é $\ell = \sqrt{\rho}\,(K-1)$, e um nível $\ell$ de um mapa de altura $H$ tem $H/2^{\ell}$ texels para 180°. Para $\rho = 0{,}56$ e $K = 6$, $\ell \approx 3{,}7$: o texel tem cerca de 19° com $H = 128$ e cerca de 4,7° com $H = 512$. Ou seja, nesta formulação $\rho$ não é uma rugosidade física, mas um índice de desfoque cuja largura angular é proporcional a $1/H$; valores de $\rho$ não são comparáveis entre configurações com $H$ diferente. Os mapas aprendidos na *bell* (Figura 7, abaixo) são coerentes com essa leitura: com $H = 512$ o mapa mostra uma estrutura de ambiente reconhecível, e com $H = 128$ apenas blocos. Como os três ajustes foram aplicados juntos, esses dados indicam mecanismos plausíveis, mas não medem a contribuição de cada ajuste.

**Tabela 6**: **NeRO: custo do treino de 30 mil iterações na RTX 3050 (6 GB).** Energia = potência média da GPU × tempo; memória = máximo do `nvidia-smi`; Gaussianas ao final do treino; render = tempo de parede para renderizar as 32 vistas de teste.

| Cena | Método | Tempo (h) | Energia (Wh) | Potência (W) | Memória (GB) | Gaussianas (mil) | Render (s) |
|---|---|---|---|---|---|---|---|
| bell | GOF | 3,59 | 170 | 47,3 | 2,58 | 107 | 24 |
| | GOF+BRDF (config. Truck) | 3,18 | 153 | 48,0 | 2,83 | 97 | 23 |
| | GOF+BRDF ajustado | 2,51 | 123 | 48,9 | 2,71 | 86 | 21 |
| | GaussianShader | 6,09 | 414 | 68,0 | 5,34 | 231 | 50 |
| cat | GOF | 2,54 | 123 | 48,4 | 2,62 | 79 | 21 |
| | GOF+BRDF (config. Truck) | 2,23 | 107 | 48,0 | 2,59 | 71 | 21 |
| | GOF+BRDF ajustado | 2,06 | 103 | 49,7 | 2,62 | 64 | 20 |
| | GaussianShader | 6,08 | 414 | 68,1 | 5,47 | 155 | 52 |
| teapot | GOF | 4,04 | 189 | 46,8 | 2,54 | 171 | 24 |
| | GOF+BRDF (config. Truck) | 3,77 | 180 | 47,7 | 2,58 | 202 | 23 |
| | GOF+BRDF ajustado | 3,39 | 165 | 48,5 | 2,64 | 196 | 23 |
| | GaussianShader | 6,47 | 439 | 67,9 | 5,55 | 383 | 50 |

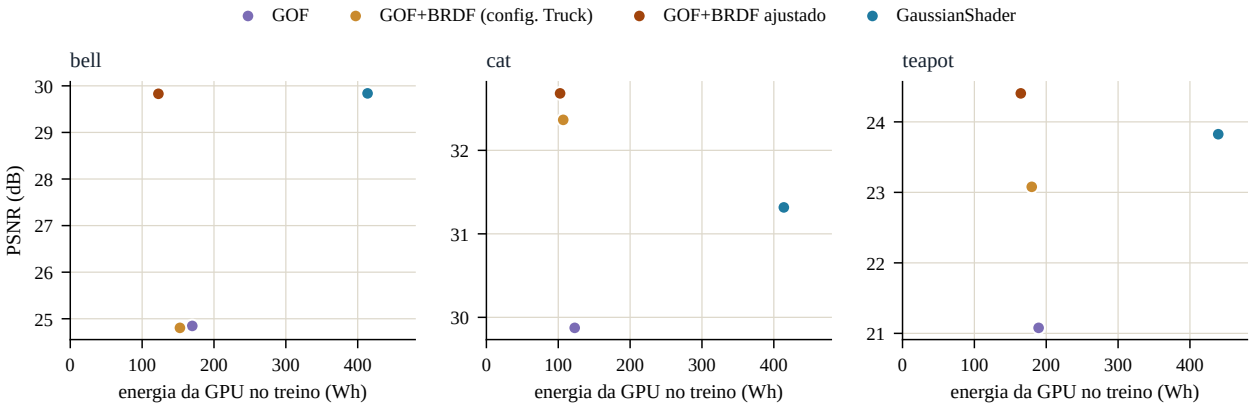



**Figura 6**: **Qualidade contra energia de treino no NeRO.** Cada ponto é um treino completo.

# 6 Discussão

**Respostas à pergunta de pesquisa.** (i) A integração reproduz o GOF quando o termo especular é nulo e o Fresnel está desligado, o que é verificado por teste e torna a comparação C1 interpretável. (ii) O termo especular melhora a síntese de vistas no Truck (+0,41 dB) e em dois dos três objetos com a configuração herdada; no objeto mais espelhado, ele só passa a ajudar depois do ajuste. (iii) Com o ajuste, o GOF+BRDF empata com o GaussianShader em PSNR na *bell*, com SSIM e LPIPS um pouco piores, e o supera nas três métricas no *cat* e no *teapot*, com menos tempo, energia e memória.

**Tabela 7**: **Material aprendido pelo GOF+BRDF no NeRO.** Estatísticas ponderadas pela opacidade das Gaussianas; tom = média dos três canais; rugosidade com mediana e quartis.

| Cena | Configuração | Tom mediano | Tom > 0,1 | Rugosidade mediana [Q1; Q3] |
|---|---|---|---|---|
| bell | config. Truck | 0,15 | 59% | 0,54 [0,34; 0,72] |
| | ajustado | 0,57 | 82% | 0,56 [0,43; 0,74] |
| cat | config. Truck | 0,12 | 54% | 0,41 [0,24; 0,66] |
| | ajustado | 0,32 | 77% | 0,67 [0,53; 0,83] |
| teapot | config. Truck | 0,23 | 66% | 0,31 [0,14; 0,58] |
| | ajustado | 0,54 | 84% | 0,58 [0,40; 0,72] |

**A configuração pode anular o termo especular.** O achado mais útil na prática é que a mesma formulação vai de "nenhum ganho" a "empate com o GaussianShader" na *bell* mudando hiperparâmetros pensados para cenas foscas. A esparsidade do tom é razoável quando poucas superfícies são especulares, mas num objeto metálico ela empurra o tom justamente para o valor que desliga o reflexo; e um mapa de 128 texels de altura limita a nitidez do reflexo qualquer que seja a rugosidade aprendida. Isso sugere tratar esses hiperparâmetros como dependentes da classe de material da cena, e não como constantes do método.

**Por que o GOF+BRDF pode superar o GaussianShader.** Não testamos a causa da vantagem no *cat* e no *teapot*. Hipóteses compatíveis com os dados são a renderização por interseção raio–Gaussiana e as perdas geométricas do GOF, que poderiam dar normais mais estáveis ao termo especular, e o Fresnel, ausente no GaussianShader. O GaussianShader termina com cerca do dobro de Gaussianas, o que torna improvável que a diferença venha de falta de capacidade do GaussianShader. Separar essas causas exigiria ablações cruzadas que não fizemos.

**Discordância entre métricas.** No Truck, PSNR e SSIM favorecem o GOF+BRDF e o LPIPS favorece o GOF; na *bell* (C3), o PSNR empata e SSIM e LPIPS favorecem o GaussianShader. O LPIPS mede semelhança de características profundas e pode penalizar padrões de alta frequência que o PSNR quase não vê; diferenças de LPIPS da ordem de 0,003 são pequenas, mas, sendo consistentes entre vistas, não devem ser descartadas.

**Relação com a literatura.** O resultado é coerente com a observação de que o ganho de modelos de sombreamento cresce com a especularidade da cena (*6*). Ele não toca as questões abertas da

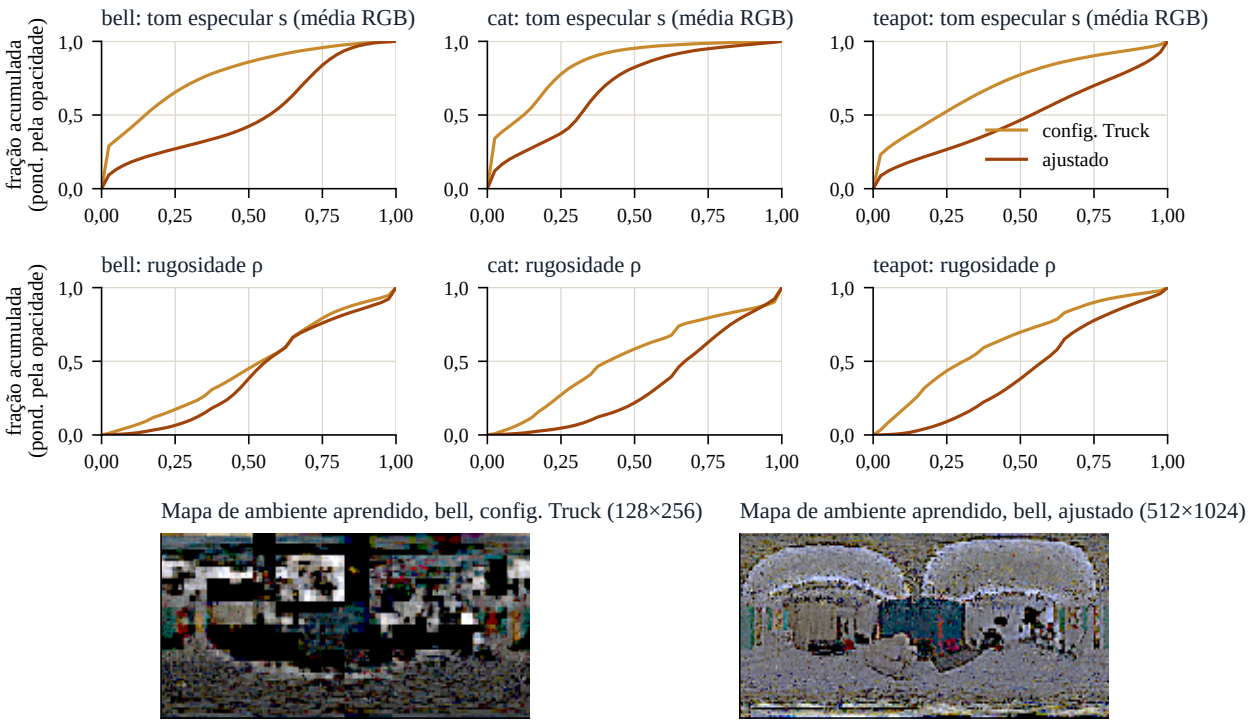



**Figura 7**: **Mecanismo do ajuste.** Distribuições acumuladas, ponderadas pela opacidade, do tom especular (primeira linha) e da rugosidade (segunda linha), para a configuração do Truck e a ajustada. Abaixo: mapa de ambiente aprendido na *bell* em cada configuração (exibição com escala no percentil 99,5 e gama 2,2).

terceira onda de trabalhos, como iluminação indireta, visibilidade e reconstrução de material com significado físico (*16, 34, 36*); nosso método ignora essas componentes por construção.

## 6.1 Ameaças à validade

**Validade interna.** Há uma semente por configuração, e o treino não é reprodutível bit a bit; os intervalos de confiança cobrem a variação entre vistas, não entre treinos. O ajuste foi escolhido na *bell*, que também é cena de teste. Os três ajustes foram aplicados juntos. O esforço de ajuste foi assimétrico: ajustamos o nosso método e usamos o GaussianShader na configuração dos autores; o fato de nossa execução do GaussianShader ficar acima do publicado em dois dos três objetos atenua, mas não elimina, essa ameaça.

**Validade externa.** Avaliamos uma cena real e três objetos sintéticos, todos com iluminação fixa no mundo. Não avaliamos geometria, relighting nem a qualidade do material recuperado, e o modo de iluminação em referencial de câmera não foi testado.

**Validade de construto.** As métricas incluem o fundo branco no NeRO; a energia é só a da GPU; a memória do NeRO vem do `nvidia-smi` e inclui reservas do alocador; o tempo de render inclui leitura e escrita de arquivos.

# 7 Limitações e trabalhos futuros

*Sombreamento por Gaussiana.* A cor é avaliada uma vez por Gaussiana e por vista, como no GaussianShader; um reflexo menor que uma Gaussiana não pode ser representado, e a normal usada não é a normal por raio do GOF. Um sombreamento por pixel com a normal por raio exigiria alterar o rasterizador. *Sem luz indireta, sem visibilidade.* Inter-reflexões e sombras ficam por conta do mapa de ambiente e do difuso. *Rugosidade não identificável e não calibrada.* Sob iluminação uniforme, $\partial \mathcal{L} / \partial \rho = 0$; e, com a pirâmide por média, $\rho$ depende de $H$ (Seção 5.4). *Memória.* Em 6 GB usamos SH de grau 0, o que desliga o termo residual; graus maiores não foram medidos. *Geometria.* A extração de malha de cenas com cerca de 2 milhões de Gaussianas não coube em 6 GB na implementação original; um processamento em lotes está em desenvolvimento, e métricas de geometria (Chamfer, F1) ficam para a próxima versão. *Próximos passos.* Repetir os treinos com ao menos três sementes; separar os três ajustes por ablação; avaliar os outros cinco objetos do NeRO e a geometria contra as malhas de referência; e validar o modo em referencial de câmera numa captura em mesa giratória. A motivação original deste estudo é a documentação digital de amostras com microestruturas brilhantes, como testemunhos de sondagem, cuja avaliação depende de autorização de uso dos dados.

# 8 Conclusão

Acoplar o sombreamento especular do GaussianShader ao GOF pelo caminho de cor pré-computada é suficiente para dar ao GOF um termo especular útil, sem alterar o rasterizador nem a extração

de malha, e com equivalência verificável ao GOF. O ganho é pequeno numa cena real predominantemente fosca e grande em objetos brilhantes, desde que a configuração não neutralize o termo especular; ajustado, o método iguala ou supera o GaussianShader nos três objetos testados em PSNR, com uma fração da energia e da memória. Esses números vêm de uma semente por configuração e de um ajuste escolhido em uma das cenas de teste, e devem ser lidos como evidência preliminar até a repetição com várias sementes e a ablação dos ajustes.

# Disponibilidade de código e dados

O código, os scripts de treino, medição de custo, métricas e estatística, e os scripts que geram todas as figuras e tabelas deste manuscrito a partir dos resultados estão em [repositório a definir]. Os dados usados são públicos (*8, 42*).

# Agradecimentos

[Preencher: financiamento, infraestrutura, contribuições de cada autor.]

# Referências

1. B. Mildenhall, *et al.*, NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis, in *European Conference on Computer Vision (ECCV)* (2020), pp. 405–421, doi:10.1007/978-3-030-58452-8_24.

2. B. Kerbl, G. Kopanas, T. Leimkuehler, G. Drettakis, 3D Gaussian Splatting for Real-Time Radiance Field Rendering. *ACM Transactions on Graphics* **42** (4), 1–14 (2023), doi:10.1145/3592433.

3. Y. Bao, *et al.*, 3D Gaussian Splatting: Survey, Technologies, Challenges, and Opportunities. *IEEE Transactions on Circuits and Systems for Video Technology* **35** (7), 6832–6852 (2025), doi:10.1109/tcsvt.2025.3538684.

4. D. Verbin, *et al.*, Ref-NeRF: Structured View-Dependent Appearance for Neural Radiance Fields, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), pp. 5481–5490, doi:10.1109/cvpr52688.2022.00541, arXiv:2112.03907.

5. P. Mézières, F. Desrichard, D. Vanderhaeghe, M. Paulin, Harmonics Virtual Lights : fast projection of luminance field on spherical harmonics for efficient rendering. *arXiv preprint arXiv:2201.01487* (2022).

6. Y. Jiang, *et al.*, GaussianShader: 3D Gaussian Splatting with Shading Functions for Reflective Surfaces, in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 5322–5332, doi:10.1109/cvpr52733.2024.00509, arXiv:2311.17977.

7. T. Wu, *et al.*, DeferredGS: Decoupled and Relightable Gaussian Splatting With Deferred Shading. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **47** (8), 6307–6319 (2025), doi:10.1109/tpami.2025.3560933.

8. Y. Liu, *et al.*, NeRO: Neural Geometry and BRDF Reconstruction of Reflective Objects from Multiview Images. *ACM Transactions on Graphics* **42** (4), 1–22 (2023), doi:10.1145/3592134.

9. Z. Li, C. Wang, P. Wang, G. Wei, Y. Zhou, PDRGS: Probabilistic Distribution Reshaping for Specular Geometry Reconstruction in 3D Gaussian Splatting. *Computer Graphics Forum* p. e70650 (2026), doi:10.1111/cgf.70650.

10. A. Guédon, V. Lepetit, SuGaR: Surface-Aligned Gaussian Splatting for Efficient 3D Mesh Reconstruction and High-Quality Mesh Rendering, in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 5354–5363, doi:10.1109/cvpr52733.2024.00512.

11. B. Huang, Z. Yu, A. Chen, A. Geiger, S. Gao, 2D Gaussian Splatting for Geometrically Accurate Radiance Fields, in *ACM SIGGRAPH 2024 Conference Papers* (2024), pp. 1–11, doi:10.1145/3641519.3657428.

12. Z. Yu, T. Sattler, A. Geiger, Gaussian Opacity Fields: Efficient Adaptive Surface Reconstruction in Unbounded Scenes. *ACM Transactions on Graphics* **43** (6), 1–13 (2024), arXiv:2404.10772, doi:10.1145/3687937.

13. Z. Liang, Q. Zhang, Y. Feng, Y. Shan, K. Jia, GS-IR: 3D Gaussian Splatting for Inverse Rendering, in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 21644–21653, doi:10.1109/cvpr52733.2024.02045.

14. Y. Shi, *et al.*, GIR: 3D Gaussian Inverse Rendering for Relightable Scene Factorization. *IEEE Transactions on Pattern Analysis and Machine Intelligence* pp. 1–12 (2025), arXiv:2312.05133, doi:10.1109/tpami.2025.3575937.

15. Y. Yao, Z. Zeng, C. Gu, X. Zhu, L. Zhang, Reflective Gaussian Splatting. *arXiv preprint arXiv:2412.19282* (2024), international Conference on Learning Representations (ICLR) 2025.

16. C. Gu, X. Wei, Z. Zeng, Y. Yao, L. Zhang, IRGS: Inter-Reflective Gaussian Splatting with 2D Gaussian Ray Tracing, in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2025), pp. 10943–10952, doi:10.1109/cvpr52734.2025.01022, arXiv:2412.15867.

17. B. Karis, Real Shading in Unreal Engine 4, SIGGRAPH 2013 Course: Physically Based Shading in Theory and Practice (2013), `https://blog.selfshadow.com/publications/s2013-shading-course/`.

18. J. Munkberg, *et al.*, Extracting Triangular 3D Models, Materials, and Lighting From Images, in *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022), pp. 8270–8280, doi:10.1109/cvpr52688.2022.00810.

19. Z. Yu, A. Chen, B. Huang, T. Sattler, A. Geiger, Mip-Splatting: Alias-Free 3D Gaussian Splatting, in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024), pp. 19447–19456, doi:10.1109/cvpr52733.2024.01839.

20. B. Zhang, *et al.*, RaDe-GS: Rasterizing Depth in Gaussian Splatting. *ACM Transactions on Graphics* **45** (2), 1–14 (2026), doi:10.1145/3789201.

21. X. Lyu, *et al.*, 3DGSR: Implicit Surface Reconstruction with 3D Gaussian Splatting. *ACM Transactions on Graphics* **43** (6), 1–12 (2024), doi:10.1145/3687952.

22. A. Guédon, *et al.*, MILo: Mesh-In-the-Loop Gaussian Splatting for Detailed and Efficient Surface Reconstruction. *ACM Transactions on Graphics* **44** (6), 1–15 (2025), doi:10.1145/3763339.

23. K. Jiang, V. Sivaram, C. Peng, R. Ramamoorthi, Geometry Field Splatting with Gaussian Surfels, in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2025), pp. 5752–5762, doi:10.1109/cvpr52734.2025.00540.

24. N. Moenne-Loccoz, *et al.*, 3D Gaussian Ray Tracing: Fast Tracing of Particle Scenes. *ACM Transactions on Graphics* **43** (6), 1–19 (2024), arXiv:2407.07090, doi:10.1145/3687934.

25. Z. Liang, H. Li, K. Jia, K. Guo, Q. Zhang, GUS-IR: Gaussian Splatting With Unified Shading for Inverse Rendering. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **47** (10), 8364–8378 (2025), doi:10.1109/tpami.2025.3578416.

26. Y. Zhou, F. Zhang, Z. Wang, L. Zhang, RTR-GS: 3D Gaussian Splatting for Inverse Rendering with Radiance Transfer and Reflection, in *Proceedings of the 33rd ACM International Conference on Multimedia* (2025), pp. 6888–6897, doi:10.1145/3746027.3755197.

27. S. Lai, *et al.*, GlossyGS: Inverse Rendering of Glossy Objects With 3D Gaussian Splatting. *IEEE Transactions on Visualization and Computer Graphics* **31** (10), 7478–7491 (2025), doi:10.1109/tvcg.2025.3547063.

28. X. Yang, M. Wei, GOGS: High-Fidelity Geometry and Relighting for Glossy Objects via Gaussian Surfels. *arXiv preprint arXiv:2508.14563* (2025).

29. Z. Zhu, B. Wang, J. Yang, GS-ROR2: Bidirectional-guided 3DGS and SDF for Reflective Object Relighting and Reconstruction. *ACM Transactions on Graphics* **45** (1), 1–19 (2025), doi:10.1145/3759248.

30. Z.-L. Zhu, J. Yang, B. Wang, Gaussian Splatting with Discretized SDF for Relightable Assets, in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)* (2025), pp. 25155–25164, doi:10.1109/iccv51701.2025.02333.

31. K. Ye, C. Gao, G. Li, W. Chen, B. Chen, GeoSplatting: Towards Geometry Guided Gaussian Splatting for Physically-Based Inverse Rendering, in *2025 IEEE/CVF International Conference on Computer Vision (ICCV)* (2025), pp. 28991–29000, doi:10.1109/iccv51701.2025.02692.

32. H. Sun, Y. Gao, J. Xie, J. Yang, B. Wang, SVG-IR: Spatially-Varying Gaussian Splatting for Inverse Rendering, in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2025), pp. 16143–16152, doi:10.1109/cvpr52734.2025.01505.

33. C. Gu, X. Wei, Z. Zeng, Y. Yao, L. Zhang, Inter-Reflective Gaussian Splatting for Robust and Efficient Inverse Rendering. *arXiv preprint arXiv:2607.22780* (2026).

34. J. Zhu, *et al.*, PTIR-GS: Path-Traced Inverse Rendering with Global Illumination in 3D Gaussian Fields. *arXiv preprint arXiv:2606.09606* (2026).

35. K. Jiang, *et al.*, Differentiable Light Transport with Gaussian Surfels via Adapted Radiosity for Efficient Relighting and Geometry Reconstruction. *ACM Transactions on Graphics* **44** (6), 1–25 (2025), doi:10.1145/3763305.

36. K. B. Han, J. Kim, W. J. Kim, J. Seo, S.-e. Yoon, Radiometrically Consistent Gaussian Surfels for Inverse Rendering. *arXiv preprint arXiv:2603.01491* (2026), international Conference on Learning Representations (ICLR) 2026.

37. H. Chen, Z. Lin, J. Zhang, GI-GS: Global Illumination Decomposition on Gaussian Splatting for Inverse Rendering. *arXiv preprint arXiv:2410.02619* (2024).

38. J. Xu, *et al.*, GS-PI: An Optimization-Decoupled Appearance Decomposition Approach for Generating PBR Gaussian Assets. *arXiv preprint arXiv:2609.19907* (2026).

39. P. Noras, J. M. Choi, D. Stricker, P. Peers, R. Sengupta, GAINS: Gaussian-based Inverse Rendering from Sparse Multi-View Captures. *arXiv preprint arXiv:2512.09925* (2025).

40. C. Schlick, An Inexpensive BRDF Model for Physically-based Rendering. *Computer Graphics Forum* **13** (3), 233–246 (1994), doi:10.1111/1467-8659.1330233.

41. B. Walter, S. R. Marschner, H. Li, K. E. Torrance, Microfacet Models for Refraction through Rough Surfaces, in *Eurographics Symposium on Rendering (EGSR)* (2007), pp. 195–206, doi:10.2312/EGWR/EGSR07/195-206.

42. A. Knapitsch, J. Park, Q.-Y. Zhou, V. Koltun, Tanks and Temples: Benchmarking Large-Scale Scene Reconstruction. *ACM Transactions on Graphics* **36** (4), 1–13 (2017), doi:10.1145/3072959.3073599.

43. J. L. Schonberger, J.-M. Frahm, Structure-from-Motion Revisited, in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016), pp. 4104–4113, doi:10.1109/cvpr.2016.445.

44. Z. Wang, A. C. Bovik, H. R. Sheikh, E. P. Simoncelli, Image quality assessment: from error visibility to structural similarity. *IEEE Transactions on Image Processing* **13** (4), 600–612 (2004), doi:10.1109/tip.2003.819861.

45. R. Zhang, P. Isola, A. A. Efros, E. Shechtman, O. Wang, The Unreasonable Effectiveness of Deep Features as a Perceptual Metric, in *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2018), pp. 586–595, doi:10.1109/cvpr.2018.00068.

46. B. Efron, Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* **7** (1), 1–26 (1979), doi:10.1214/aos/1176344552.

47. F. Wilcoxon, Individual Comparisons by Ranking Methods. *Biometrics Bulletin* **1** (6), 80 (1945), doi:10.2307/3001968.

48. E. Strubell, A. Ganesh, A. McCallum, Energy and Policy Considerations for Deep Learning in NLP, in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (2019), pp. 3645–3650, doi:10.18653/v1/p19-1355.

# Material Suplementar

## S1 Comparações adicionais no Truck

A Tabela S1 inclui dois sistemas treinados em outra máquina, com outros hiperparâmetros: o GaussianShader com sua configuração própria e o GOF com SH de grau 3 e aparência desacoplada. As imagens de referência das três origens são idênticas pixel a pixel, mas as diferenças de configuração impedem atribuir causa às diferenças entre esses sistemas e os nossos.

**Tabela S1**: **Truck: comparação não controlada.** †Treinado em outra máquina, com hiperparâmetros próprios; não comparável de forma causal.

| Sistema | PSNR (dB) ↑ | SSIM ↑ | LPIPS ↓ |
|---|---|---|---|
| GOF (este trabalho) | 25,220 | 0,8863 | 0,1338 |
| GOF+BRDF (este trabalho) | 25,625 | 0,8894 | 0,1363 |
| GaussianShader† | 23,836 | 0,8496 | 0,1829 |
| GOF, SH grau 3 + aparência desacoplada† | 23,393 | 0,8960 | 0,1182 |

## S2 Estatísticas completas do NeRO

**Tabela S2**: **NeRO: todos os pares.** Média [IC 95%] da diferença "primeiro − segundo"; $p$ de Wilcoxon para o PSNR (os de SSIM e LPIPS estão em `dados/nero/*_analysis.json`).

| Cena | Par | ΔPSNR (dB) | ΔSSIM | ΔLPIPS | $p$ (PSNR) |
|---|---|---|---|---|---|
| bell | GOF+BRDF − GOF | −0,04 [−0,26; +0,18] | −0,0115 [−0,0131; −0,0100] | +0,0050 [+0,0040; +0,0060] | 0,586 |
| | GOF+BRDF ajustado − GOF | +4,98 [+4,79; +5,18] | +0,0307 [+0,0285; +0,0330] | −0,0242 [−0,0259; −0,0226] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF | +4,99 [+4,75; +5,23] | +0,0362 [+0,0331; +0,0393] | −0,0275 [−0,0293; −0,0256] | $4{,}7 \times 10^{-10}$ |
| | GOF+BRDF ajustado − GOF+BRDF | +5,02 [+4,81; +5,26] | +0,0422 [+0,0388; +0,0456] | −0,0292 [−0,0312; −0,0272] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF+BRDF | +5,03 [+4,77; +5,31] | +0,0477 [+0,0437; +0,0517] | −0,0325 [−0,0345; −0,0304] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF+BRDF ajustado | +0,01 [−0,23; +0,24] | +0,0055 [+0,0037; +0,0073] | −0,0033 [−0,0046; −0,0019] | 0,705 |
| cat | GOF+BRDF − GOF | +2,49 [+2,27; +2,69] | +0,0144 [+0,0137; +0,0152] | −0,0179 [−0,0189; −0,0169] | $4{,}7 \times 10^{-10}$ |
| | GOF+BRDF ajustado − GOF | +2,81 [+2,63; +2,99] | +0,0177 [+0,0166; +0,0190] | −0,0229 [−0,0246; −0,0213] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF | +1,44 [+1,12; +1,75] | +0,0086 [+0,0073; +0,0097] | −0,0142 [−0,0154; −0,0130] | $9{,}6 \times 10^{-8}$ |
| | GOF+BRDF ajustado − GOF+BRDF | +0,32 [+0,21; +0,42] | +0,0034 [+0,0027; +0,0040] | −0,0050 [−0,0060; −0,0041] | $1{,}1 \times 10^{-6}$ |
| | GaussianShader − GOF+BRDF | −1,05 [−1,25; −0,86] | −0,0058 [−0,0069; −0,0047] | +0,0037 [+0,0027; +0,0047] | $9{,}3 \times 10^{-10}$ |
| | GaussianShader − GOF+BRDF ajustado | −1,37 [−1,60; −1,15] | −0,0092 [−0,0107; −0,0077] | +0,0087 [+0,0075; +0,0100] | $4{,}7 \times 10^{-10}$ |
| teapot | GOF+BRDF − GOF | +2,00 [+1,61; +2,40] | +0,0098 [+0,0084; +0,0112] | −0,0050 [−0,0059; −0,0041] | $9{,}3 \times 10^{-10}$ |
| | GOF+BRDF ajustado − GOF | +3,33 [+3,07; +3,59] | +0,0272 [+0,0261; +0,0283] | −0,0112 [−0,0121; −0,0103] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF | +2,75 [+2,47; +3,01] | +0,0231 [+0,0217; +0,0245] | −0,0087 [−0,0100; −0,0073] | $4{,}7 \times 10^{-10}$ |
| | GOF+BRDF ajustado − GOF+BRDF | +1,33 [+1,14; +1,52] | +0,0174 [+0,0160; +0,0187] | −0,0061 [−0,0068; −0,0055] | $4{,}7 \times 10^{-10}$ |
| | GaussianShader − GOF+BRDF | +0,75 [+0,54; +0,95] | +0,0132 [+0,0123; +0,0142] | −0,0036 [−0,0043; −0,0030] | $2{,}5 \times 10^{-7}$ |
| | GaussianShader − GOF+BRDF ajustado | −0,58 [−0,67; −0,49] | −0,0041 [−0,0051; −0,0032] | +0,0025 [+0,0018; +0,0032] | $4{,}7 \times 10^{-10}$ |

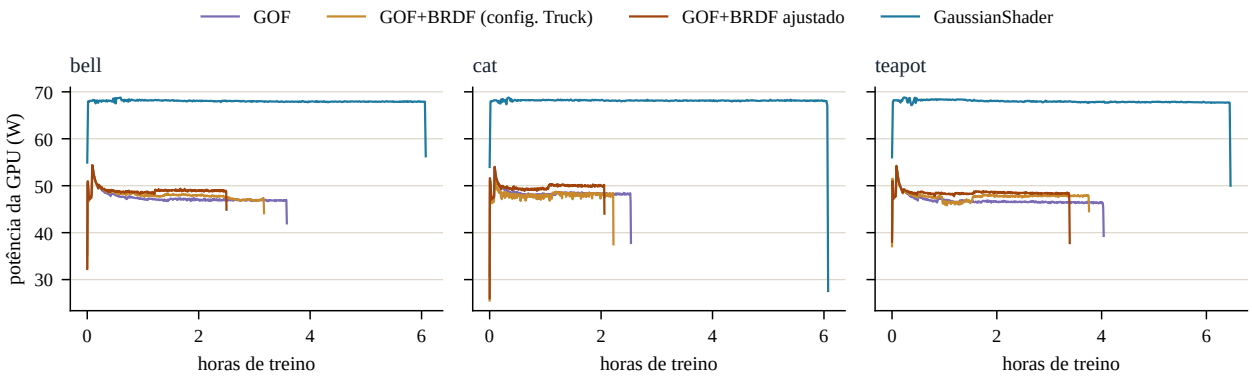



**Figura S1**: **Potência da GPU ao longo do treino** (média móvel das amostras de 2 s).

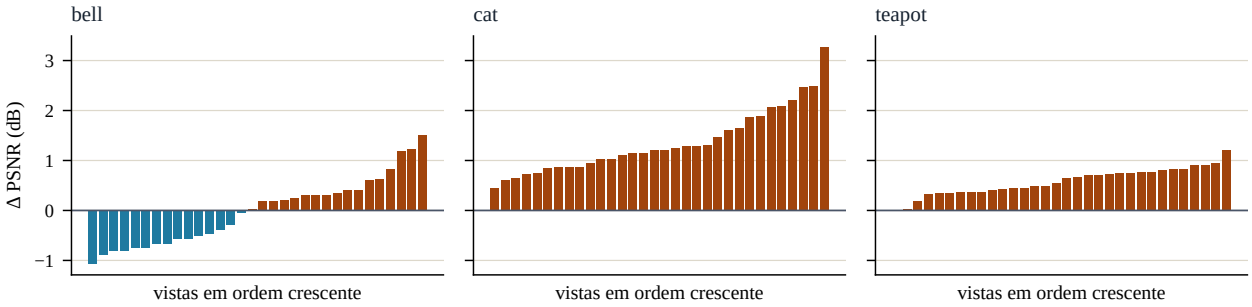



**Figura S2**: **Diferença de PSNR por vista, GOF+BRDF ajustado menos GaussianShader**, ordenada (laranja: GOF+BRDF ajustado melhor; azul: GaussianShader melhor).

# S3 Testes e modos de falha silenciosos

A suíte automatizada tem 36 testes, executados na GPU, que passam na versão usada nos experimentos. Além das igualdades da Seção 3.5, ela cobre propriedades da reflexão (involução e preservação de norma), os limites do Fresnel (refletância **s** em incidência normal e 1 em ângulo rasante), o efeito do referencial da iluminação e a orientação da normal. Cinco falhas encontradas durante o desenvolvimento não geravam erro nem mudavam o formato dos tensores, e por isso só foram detectadas comparando o comportamento com o esperado; cada uma tem hoje um teste de regressão.

1. *Material desvinculado do otimizador na poda.* Ao podar Gaussianas, o tensor de material era recriado a partir do valor antigo; o modelo passava a ler um tensor e o otimizador a atualizar outro, e o material congelava na primeira poda.

2. *Material ausente do* checkpoint. Retomar um treino recriava o material vazio.

3. *Material ausente do arquivo de pontos final.* A avaliação, que recarrega o modelo desse arquivo, usaria material padrão, diferente do treinado.

4. *Iluminação ausente do* checkpoint. Retomar um treino reiniciaria o mapa de ambiente aprendido, mantendo o estado do otimizador calculado para o mapa antigo.

5. *Gradiente infinito nos polos do mapa.* O primeiro treino real com o termo especular foi a `NaN` dez iterações após a ativação do ramo especular; a correção é o afastamento $\varepsilon$ da Seção 3.

# S4 Iluminação em referencial de câmera (não validada)

Numa mesa giratória, o objeto gira e a câmera e as luzes ficam paradas. Sejam $R_i$ a rotação (desconhecida) da mesa na imagem $i$ e $C$ a rotação constante da sala para a câmera. O *structure-from-motion* vê o objeto parado e entrega a rotação mundo–câmera $W_i = C\,R_i$. Uma direção de reflexão $\mathbf{r}_O$ em coordenadas do objeto corresponde, na sala, a

$$\mathbf{r}_{\text{sala}} = R_i\,\mathbf{r}_O = C^{-1}(CR_i)\,\mathbf{r}_O = C^{-1}W_i\,\mathbf{r}_O = C^{-1}\mathbf{r}_{\text{câm}}, \tag{S1}$$

em que $\mathbf{r}_{\text{câm}}$ é a reflexão em coordenadas da câmera. Portanto $L(\mathbf{r}_{\text{sala}}) = L'(\mathbf{r}_{\text{câm}})$, com $L' = L \circ C^{-1}$: indexar um mapa aprendido pela reflexão em coordenadas da câmera equivale a aprender o mapa da sala pré-rotacionado por uma rotação global constante, sem estimar o ângulo da mesa. A dedução exige que a orientação da câmera seja a mesma em todas as fotos (translações são permitidas); com várias orientações seria necessária uma rotação por estação. Este modo está implementado, com um teste que verifica apenas que a rotação é aplicada; nenhum experimento deste artigo o utiliza.